# /db

Session database: /db prune [–days N] [–dry-run] deletes old messages (sessions are kept) | /db help

## Overview

`/db` is the operator surface for the SQLite session store. It has exactly two subcommands and no hidden behaviour: `prune` ages out chat messages, `help` prints the usage page (a bare `/db` prints the same page).

`/db prune` deletes **messages** older than a retention window. The **session row is kept**, so an expired conversation stays listed in `ragent session list` and can be resumed - it simply reads back with an empty transcript. This is the difference between `/db prune` and a session delete.

The same operation is available headless as `ragent db prune [--days N] [--dry-run]`, and the identical prune runs automatically at startup when `storage.retention.enabled` is set in `ragent.json`.

Nothing is deleted without confirmation: a real prune opens a `Delete messages older than N day(s)?` dialog that starts on `No`, so a stray `Enter` cannot remove messages.

## Syntax

```
/db
/db help
/db prune
/db prune --days 30
/db prune --days=30
/db prune --dry-run
/db prune --dry-run --days 30
```

## Options / Subcommands

| Form | Description |
|---|---|
| `/db` or `/db help` | Print the `/db` help page (also `--help` / `-h`). |
| `/db prune` | Delete messages older than the configured window, after confirmation. |
| `/db prune --days N` | Delete messages older than `N` days, after confirmation. |
| `/db prune --days=N` | Joined form of the same flag; identical result. |
| `/db prune --dry-run` | Report how many messages would be deleted; delete nothing. |

Flag rules:

- `--days` accepts both the two-token (`--days 30`) and the joined (`--days=30`) form. The value must be a **positive** day count.
- `--days 0` is **refused**: a zero-day window would empty the store on every run, so it is treated as a usage error rather than silently accepted.

- Omit `--days` and the window is `storage.retention.message_days` from `ragent.json` (default `14`).
- `--dry-run` composes with `--days` in either order. It never deletes, and it never arms the confirmation dialog.
- Any unrecognised argument (for example `--wat`) fails with a message naming the offending token and the usage line; nothing is deleted.

## Examples

Count the messages older than 30 days without deleting anything:

```
/db prune --dry-run --days 30
```

```
From: /db prune --dry-run

Would delete **128** message(s) older than 30 day(s); sessions are kept.
Nothing was deleted.
```

Prune with the confirmation dialog (default window from config):

```
/db prune
```

```
 Delete messages older than 14 day(s)?
   [ Yes ]   [ No ]
```

The dialog opens on `No`. `Left` / `Right` (or `Tab`) move the selection, `Enter` applies it, and `Esc` cancels - cancelling leaves the database unchanged.

Prune with an explicit 14-day window:

```
/db prune --days 14
```

After confirming:

```
From: /db prune

Deleted **128** message(s) older than 14 day(s). Sessions are kept, so an
expired transcript stays listed but empty.

The database file does not shrink until a `VACUUM`, which needs exclusive
access - stop every other ragent process first.
```

The headless equivalent:

```
ragent db prune --days 14 --dry-run
ragent db prune --days 14
```

Read the help page:

```
/db
```

## Output

- An assistant message bubble titled `From: /db prune` (or `From: /db prune --dry-run`, `From: /db help`).

- Status bar shows one of `db: help`, `db: usage`, `db: pruned N`, `db: would prune N`, `db: prune failed`, `db: count failed`, `db: bad window`.
- No permission prompt is raised and no LLM call is made.
- A successful real prune also writes an `Info` row (`db: pruned N message(s) older than D day(s)`) to the log panel.

The prune itself is a single transaction over three tables - `messages_fts` rowids, `message_fts_keys`, then `messages` - so a failure cannot leave the contentless full-text index pointing at rows that no longer exist. The cutoff uses the fallible `chrono::Duration::checked_sub_signed`, so an extreme window returns an error rather than panicking.

## Pruning automatically at startup

Set the opt-in `storage.retention` section in `ragent.json`:

```jsonc
{
  "storage": {
    "retention": {
      "enabled": true,
      "message_days": 14
    }
  }
}
```

| Field | Type | Default | Description |
|---|---|---|---|
| `enabled` | `bool` | `false` | Run the prune at startup and allow `/db prune`. |
| `message_days` | `u32` | `14` | Window, in days, a message survives. Must be `>= 1`. |

With no `storage` section no message is ever expired. The startup pass runs on a blocking thread so the delete never stalls the startup critical path.

## Caveats

- **The file does not shrink.** `DELETE` returns pages to SQLite's freelist; `ragent.db` keeps its size until a `VACUUM` rewrites it. `VACUUM` needs exclusive access: stop every other ragent process (and release the `-wal` / `-shm` sidecars) first.
- **`--days 0` is an error, not a shortcut.** Use a real window; to delete everything deliberately, delete the sessions instead.
- **Session rows survive by design.** If you want the conversation gone from the session list too, that is a session delete, not a prune.

## Related

- `ragent db prune [--days N] [--dry-run]` - the headless equivalent
- `ragent session list` / `ragent session resume <id>` - the sessions a prune leaves intact
- `/history` - browse and re-use previous inputs

- `docs/sqlite.md` - pragma table, `journal_size_limit`, and the retention section
- `docs/howtos/config.md` - the full `storage.retention` reference